

A10_CAT-011

 **Video Explanation:**  **b-13-a-10-cat-11.mkv**

SkillSwap

Freelance Micro-Task Platform

Live Link: taskhive-eight-phi.vercel.app

If you have any query, Knock me on WhatsApp

WhatsApp: 01625337883

Test Accounts & Credentials

Field	Credential
Admin Email:	admin1@taskhive.com
Admin Pass:	admin1@taskhive.com
Freelancer Email:	freelanceruser3@gmail.com
Freelancer Pass:	freelanceruser3@gmail.com

SECTION 01: Project Overview and Discussion

Project Name: SkillSwap — Freelance Micro-Task Platform

Roles: Client · Freelancer · Admin

Payment: Stripe Checkout

Auth: BetterAuth (Credential + Google OAuth)

What is SkillSwap?

SkillSwap is a marketplace website. Here, **clients** post small, simple tasks like making a logo, writing an article, or fixing a CSS bug. **Freelancers** can look at these tasks, send applications (proposals), and get hired to finish them. It is a simpler version of Fiverr or Freelancer.com for fast, one-time jobs.

Why Should We Build It?

- Many people need quick help with small online tasks but do not have the money for big agencies.
- Freelancers need an easy place to show their work and make money from quick, small jobs.
- This project proves your full-stack coding skills: login, roles, Stripe payments, CRUD operations, and user dashboards.
- Online freelancing is growing fast, so websites like this are very useful and popular.

How Does the System Work?

1. A client posts a task with a title, description, category, budget, and deadline date.
2. Freelancers check the open tasks and apply by entering their price, completion time, and a message.
3. The client checks the applications and chooses the best freelancer for the job.
4. The client pays using Stripe Checkout. The payment must be successful before work starts.
5. The freelancer does the work, and the client marks the task as complete.
6. The client gives a star rating and a short review to the freelancer.
7. The admin checks all users, tasks, and history to keep the website safe.

Example Websites for Reference

- **Fiverr.com** — Marketplace for freelance services
- **Freelancer.com** — Project job bidding website
- **PeoplePerHour.com** — Hourly and project freelance work

SECTION 02: Requirements to Get 100% Marks

You must do everything below. If you miss anything, you will lose marks directly.

- ✓ Make at least **20 good git commits** on the frontend code with clear text messages.

- ✓ Make at least **12 good git commits** on the backend code with clear text messages.
- ✓ Write a clear **README.md file** with: project name, purpose, live website link, key features, and npm packages used.
- ✓ Keep all **frontend secret keys** safe inside an environment variables file (.env.local).
- ✓ Keep your **MongoDB login details** safe using environment variables on the backend server.
- ✓ Keep your **Better Auth secret keys** (secret, Google keys) safe using environment variables.
- ✓ Make the layout **fully responsive** — it must look good on mobile, tablet, and desktop screens.
- ✓ Create a professional **UI/UX layout** — use clean colors, proper text spacing, and neat spacing.
- ✓ **Do not copy text or projects.** If your project looks exactly like course modules or past tasks, you will get 0 marks.
- ✓ Add proper **error messages** on all input forms, backend endpoints, and router paths.
- ✓ Fix all **page reload bugs** — reloading the page should never crash or break private dashboard views.

SECTION 03: Deployment Guidelines

Warning: If your deployed website does not work, you will get **0 marks** and lose out on future rewards.

! Logged-in users must **NOT** be pushed back to the Login page when they refresh a private page.

! Pages must **NOT** show any error messages when refreshed on any route path.

- ✓ Your live link must open perfectly — no empty white screens on the first load.
- ✓ No CORS errors — your backend server must accept API requests from your deployed frontend website link.
- ✓ No 404 errors — all paths must load up correctly in your production build.
- ✓ No 504 timeout errors — your server must respond quickly under load.
- ✓ Refreshing any page (even dashboard pages) must not show any error messages.
- ✓ Private routes must load perfectly even after clicking the browser refresh button.
- ✓ Logged-in users must stay logged in until they click the Logout button.

SECTION 04: Layout & Page Structure

Navbar Links

Public Links (For Everyone)

- Home
- Browse Tasks

- Browse Freelancers
- Login

Private Links (Logged-in Only)

- Dashboard
- Profile
- Logout

Footer Content Rules

- Website Logo with name
- Navigation links to main pages
- Social media links (use the new X icon, not the old blue bird)
- Copyright year text
- Email or contact info

All Required Pages

Public Pages

- Home Page
- Browse Tasks Page
- Task Details Page
- Browse Freelancers Page
- Freelancer Public Profile
- Login & Register Page
- Custom 404 Error Page

Private Pages

- Client Dashboard: Post Task, My Tasks, Proposals
- Freelancer Dashboard: My Proposals, Earnings, Profile
- Admin Dashboard: Manage Users, Manage Tasks, Transactions
- /payment/success Page (Stripe Return)

SECTION 05: Home Page Requirements

All dynamic sections must load real data from your database — do not use fake placeholder data.

Hero Banner Section

- Main Title: "Get your tasks done by skilled freelancers"
- Short sub-text explaining how the marketplace works
- Two action buttons: **Post a Task** (for clients) and **Browse Tasks** (for freelancers)

- Add smooth and clean layout animations to make the section feel alive.

Dynamic Section 1 — Latest Featured Tasks

Show the latest open tasks loaded directly from your database collection. Each card must show:

- Task title
- Client name
- Category name
- Budget size (USD)
- Due date deadline

Dynamic Section 2 — Top Freelancers

Show the best freelancers on your website platform. Each card must show:

- Freelancer name
- Profile picture image
- Skills list (tags)
- Average star rating
- Total number of finished jobs

Extra Page Sections — Pick Any Two

- **How It Works** — Easy 3-step guide: Post a Task → Get Proposals → Hire and Pay
- **Popular Categories** — Clickable block links: Design, Writing, Development, Marketing, Other
- **Platform Statistics** — Show total count of tasks posted, total users, and total payout completed
- **Testimonials** — Review block quotes from happy clients and freelancers
- **Community Highlights** — Feature box for the top worker of the current month

Animation Requirement: Include nice, smooth visual transitions or fading layouts on actions to guarantee a modern user experience.

SECTION 06: Authentication System

Authentication Library: Use **Better Auth** for login and registration. You must support email/password forms and Google OAuth sign-in.

Login Page

- Input Fields: Email, Password
- Features:
 - Email and password form login logic
 - Google OAuth login button click

- Path router redirects: Clients go to the Home page; Freelancers and Admins go straight to their Dashboard path

Register Page

- Input Fields: Name, Email, Image URL, Password
- Account Role Rules:
 - **Google OAuth Login:** Any user who signs up or logs in using the Google button is automatically saved as a **Client**.
 - **Form Registration:** Users who want to become a **Freelancer** must sign up using the standard Email/Password input form and select the **Freelancer** option checkbox/radio button manually.
- Password constraints:
 - At least 6 characters long
 - At least one capital letter
 - At least one lowercase letter

SECTION 07: Client Dashboard

Route URL Path: /dashboard/client

Only accessible by accounts with the **Client** role. Uses a responsive sidebar. Wrong user roles get sent back to /login if the user is not signed in. If the user is signed in then redirect to home route or the user's own dashboard.

Dashboard Main Statistics

Total Tasks	Open Tasks	Tasks In Progress	Total Spent (USD)

Post a Task Form

An easy input form for clients to publish a new job block on the website.

- Input Fields: Task Title, Category, Description, Budget (USD), Deadline Date
- *Saved into the tasks database collection. Default state text: **open**.*

My Tasks View

- View all your posted tasks with their live status label: Open, In Progress, Completed
- Edit task content data text while the live status is still set to Open
- Delete a task row — only possible if no proposal has been approved yet

Manage Proposals View

Shows all proposals submitted by freelancers for your job posts.

- Read application text: Freelancer name, budget price, completion days, and message note
- Click Accept → Redirect to the dummy Stripe checkout page at /payment/checkout to enter card details. Upon payment success, task status updates to "In Progress".
- Click Reject → Changes proposal status value to Rejected for that user
- Only one proposal can be accepted for a single task row

SECTION 08: Freelancer Dashboard

Route URL Path: /dashboard/freelancer

Only accessible by accounts with the **Freelancer** role. Uses a sidebar.

Dashboard Main Statistics

Total Proposals	Pending Proposals	Accepted Proposals	Total Earnings (USD)

Browse Tasks View

- Look at all open jobs posted by platform clients
- See the task title, category name, budget size, deadline date, and client name on cards
- Click on a single card to open details and send a proposal form
- A freelancer can only submit one proposal application per task card

Submit a Proposal Form

A short form shown inside the Task Details view for logged-in freelancers.

- Input Fields: Task ID, Freelancer Email, Proposed Budget (USD), Estimated Days, Cover Note Message
- *Saved into the proposals database collection. Default state text: **pending**.*

My Proposals Page

A complete table list of your sent applications.

- The page must only show proposals submitted by the logged-in freelancer, filtered using the freelancerEmail=mine query parameter on the backend.
- **Table Columns:** Task Title | Budget Bid | Date Sent | Status Text (Pending / Accepted / Rejected)

Active Projects Page

A dedicated page for freelancers to track their active in-progress or completed projects.

- View tasks where the freelancer's proposal was accepted and status is In Progress.
- **Submit Deliverable:** Click button to open a modal dialog, enter a deliverable URL (Docs, GitHub, etc.), and mark the task as Completed (updates task status to Completed and saves the deliverable_url).
- View completed projects with their submitted deliverable links.

My Earnings Page

A complete breakdown list of finished tasks and payments received.

- **Table Columns:** Task Title | Client Name | Amount Made | Completion Date

Edit Profile Page

Freelancers can change their public details here anytime.

- Input Fields: Name, Profile Photo Link, Skills (tags), Bio text, Hourly Rate (USD)

SECTION 09: Admin Dashboard

Route URL Path: /dashboard/admin

Only accessible by an account with the **Admin** role. Admin logins are hardcoded into the database. There is no user registration form for admin accounts.

Overview Statistics

Total Users	Total Tasks	Total Revenue (USD)	Active Tasks

Manage Users Page

- View list of all platform accounts: Name, email address, role, and block status
- Click Block button — blocked accounts lose login permissions completely
- Click Unblock button — reactivate old user account lines

Manage Tasks Page

- Look up every task item on the website with its current live status label
- Delete task rows if they break platform safety guidelines or use bad text

Transactions History View

Show a clear history table of all active Stripe payments processed.

- **Table Columns:** Client Email | Freelancer Email | Payout Size | Payment Date | Payment

Status Label

SECTION 10: Payment System (Stripe)

Payment Action Flow: The client pays via standard Stripe Checkout right after clicking accept on an application. Uses server checkout functions and a secure backend /confirm-session endpoint to double-check transactions before database saves.

Payment Setup:

Follow the module where we have teach you how to setup stripe

Payment Success Page Layout Requirements

- Clear success transaction confirmation message text
- Display paid task title, worker name, and price size
- An easy navigation button labeled "Go to Dashboard"

SECTION 11: UI Design Requirements

General Website Design

- ✓ Original UI design — do NOT build an exact clone of course lectures
- ✓ Identical heading font family, sizing, and colors across layout templates
- ✓ Uniform button styles for action steps site-wide
- ✓ Equal height rules on card templates inside grid columns
- ✓ Balanced contrast palette configurations
- ✓ Responsive layout for mobile, tablet, and desktop viewports
- ✓ Use the modern black X logo instead of the old bird graphic

Dashboard Interface Design

- ✓ Sidebar navigation layout blocks for client, worker, and admin pages
- ✓ Mobile sidebar slide transitions or toggles for portable viewports
- ✓ Visual layout charts or overview numbers on main views
- ✓ Profile summary card in headers or sidebar blocks
- ✓ Harmonized color accents on active dashboard steps
- ✓ Max width dashboard rows

Design Idea Sites for Help

- **uiverse.io** — Open-source, premade HTML/CSS components
- **devmeetsdevs.com** — Interface layout reference layouts
- **themeforest.net** — UI design structure research references (no cloning)
- **bootcamp.uxdesign.cc** — Asset files and free web design guides

- **Shadcncomponents (If you use shadcn)**

SECTION 12: Challenge Requirements

You must build all 3 assignment tasks below. They test data filtration, login state protection, and query offset loops.

Challenge 1: Basic Task Search & Category Filtering

- Part A: Basic Title Search
 - Place a basic text search bar inside the Browse Tasks page view to find matching items using the task title.
 - Task search data lists must render directly on the page layout view without invoking standard manual page updates.
- Part B: Simple Category Filtering
 - Put a clear dropdown box selecting specific tasks by category strings: Design, Writing, Development, Marketing, or Other.
 - Bind search logic inputs alongside dropdown state models so filters alter active task collections together.

Challenge 2: JWT Authentication and Role-Based Middleware

- Generate an encrypted JWT token upon correct login forms and bind it into secure HTTPOnly cookie properties.
- Block unauthorized connection requests on backend paths by matching incoming validation request cookies.
- Secure active layout contexts below the /dashboard/* pattern using Next.js route validation guards.
- Direct empty auth requests to /login, and respond with explicit 403 Forbidden codes if user roles mismatch.

Challenge 3: Server-Side Pagination on Browse Tasks

- Set up backend-driven data pagination controls directly inside your Browse Tasks component file.
- Limit default data query operations to fetch a maximum count of 9 task documents on single query limits.
- Build clear pagination index numbers featuring functional Previous and Next state button click elements.
- Keep current pagination calculations correct when search text words or category items change at the same time.
- Build the backend route calculation function blocks to handle incoming request parameters for page and limit.

SECTION 13: Database Architecture (MongoDB)

You must implement the following 5 user collections. Organize field parameters using these identifiers.

users

- name, email, image, role, skills, bio, isBlocked, createdAt

tasks

- title, category, description, budget, deadline, client_email, status, deliverable_url, createdAt

proposals

- task_id, freelancer_email, proposed_budget, estimated_days, cover_note, status, submitted_at

payments

- client_email, freelancer_email, task_id, amount, transaction_id, payment_status, paid_at

reviews

- task_id, reviewer_email, reviewee_email, rating, comment, created_at

SECTION 14: Optional Requirements

Bonus Options (Not mandatory, but makes your project stand out):

- **Dark / Light theme toggle** — a global look switch that saves user choice settings permanently
- **Real-time notifications** — show instant alert messages to workers when task proposals change states
- **Task bookmarking** — let workers save interesting open jobs to an internal reading list for later applications
- **Freelancer verification badge** — let admin flag verified workers with a graphic icon checkmark on their profile
- **Client rating system** — let freelancers leave star counts and review descriptions on client logs after closing a task

SECTION 15: What to Submit

Field	Value

Admin Email	admin1@taskhive.com
Admin Password	admin1@taskhive.com
Live Site Link	<u>https://taskhive-eight-phi.vercel.app/</u>
GitHub (Client)	<u>https://github.com/username/skillswap-client</u>
GitHub (Server)	<u>https://github.com/username/skillswap-server</u>